permissions to a third party. Some RBAC systems give users some discretion over which roles they use when performing API operations. For example, the same user may be able to send messages to a chatroom as themselves or using their role as Chief Financial Officer when they want to post an official statement. The NIST (National Institute of Standards and Technology) standard RBAC model (https://csrc.nist.gov/projects/role-based-access-control) includes a notion of *session*, in which a user can choose which of their roles are active at a given time when making API requests. This works similarly to scoped tokens in OAuth, allowing a session to activate only a subset of a user's roles, reducing the damage if the session is compromised. In this way, RBAC also better supports the principle of least privilege than groups because a user can act with only a subset of their full authority.

8.2.1 Mapping roles to permissions

There are two basic approaches to mapping roles to lower-level permissions inside your API. The first is to do away with permissions altogether and instead to just annotate each operation in your API with the role or roles that can call that operation. In this case, you'd replace the existing `requirePermission` filter with a new `requireRole` filter that enforced role requirements instead. This is the approach taken in Java Enterprise Edition (Java EE) and the JAX-RS framework, where methods can be annotated with the `@RolesAllowed` annotation to describe which roles can call that method via an API, as shown in listing 8.4.

Listing 8.4 Annotating methods with roles in Java EE